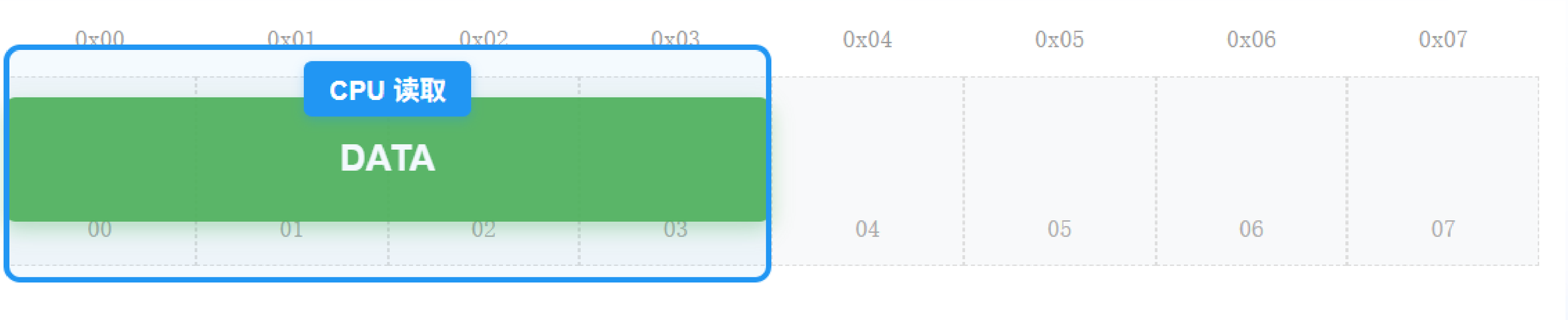

0号位
1号位
2号位
3号位
4号位
5号位
6号位
7号位

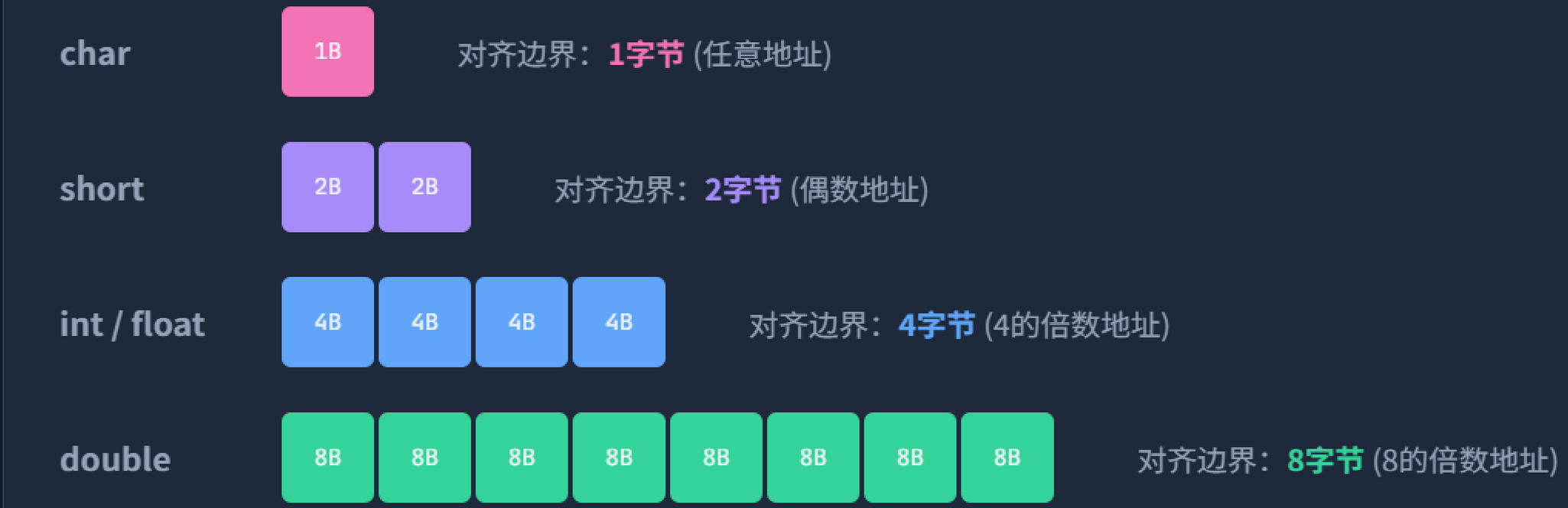

CPU
处理器
数据总线 (32/64位)
一次读取 4/8 字节
内存 Memory
0x00
0x01
0x02
0x03
D0
D1
D2
D3
✓对齐的int：一次读取完成


0x00
0x01
0x02
0x03
0x04
0x05
0x06
0x07
CPU 读取
DATA
00
01
02
03
04
05
06
07


0x00
第1次读取 (取头)
0x03
0x04
0x05
0x06
0x07
DATA
第2次读取 (取尾)
00
01
02
03
04
05
06
07

对齐规则一览

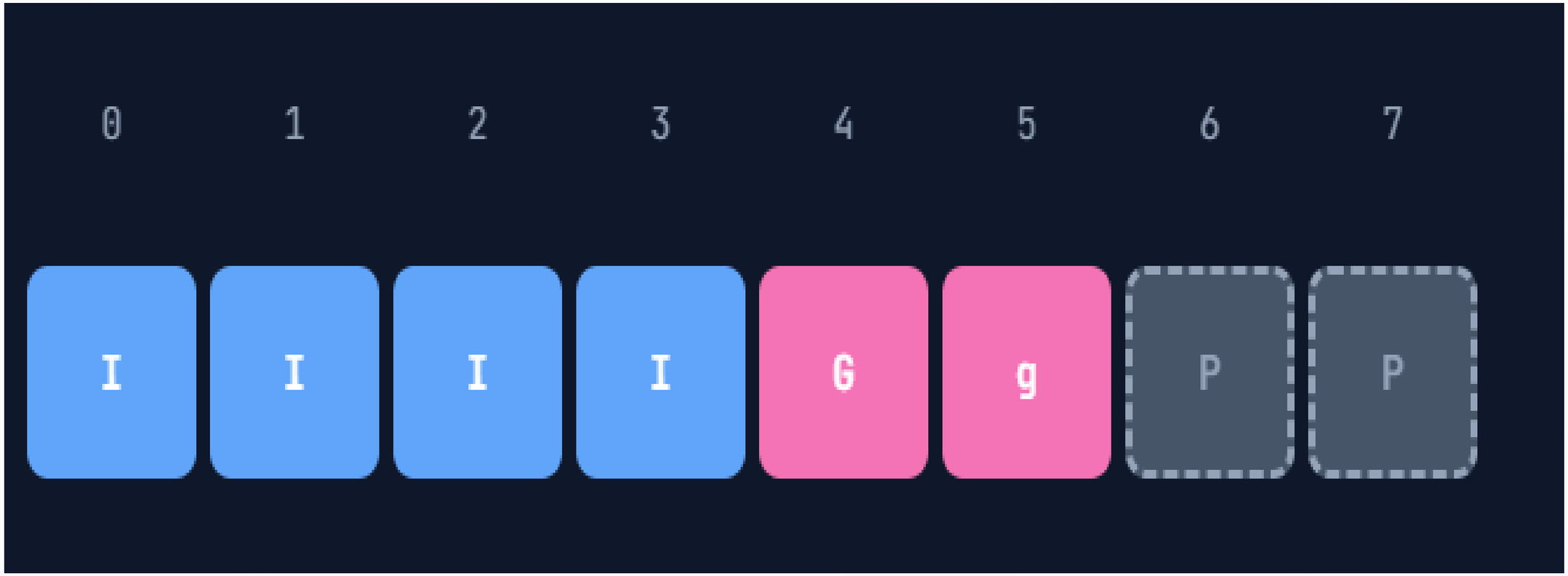
不同数据类型的对齐要求



```
struct BadStudent {
    char    grade;    // 1字节
    int     id;       // 4字节
    char    gender;   // 1字节
};
// 预期: 1+4+1 = 6字节
// 实际: 12字节!
```


0
1
2
3
4
5
6
7
G
P
P
P
I
I
I
I
8
9
10
11
g
P
P
P

```
struct GoodStudent {
    int     id;         // 4字节
    char    grade;      // 1字节
    char    gender;     // 1字节
};
// 实际: 8字节!
// 节省: 4字节 (33%)
```

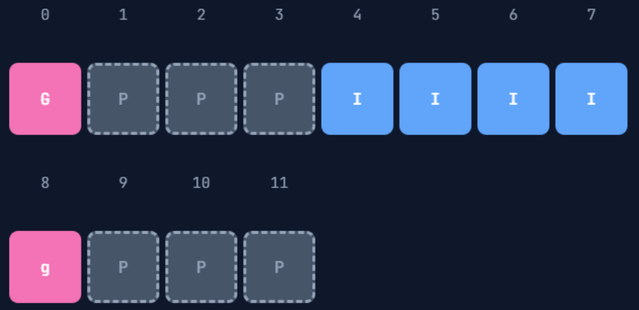

0
1
2
3
4
5
6
7
I
I
I
I
G
g
P
P

❌ 未优化的排列

浪费空间

```
struct BadStudent {
  char   grade;   // 1字节
  int    id;      // 4字节
  char   gender;  // 1字节
};
// 预期: 1+4+1 = 6字节
// 实际: 12字节!
```

12 字节



✅ 优化后的排列

节省50%

```
struct GoodStudent {
  int    id;      // 4字节
  char   grade;   // 1字节
  char   gender;  // 1字节
};
// 实际: 8字节!
// 节省: 4字节 (33%)
```

8 字节



char (1字节) int (4字节) 填充/Padding (浪费)